

in equation (9.1.4). Since each application of (9.2.4) requires two function evaluations, the actual order of the method is $\sqrt{2}$, not 2; but this is still quite respectably superlinear: The number of significant digits in the answer approximately *doubles* with each two function evaluations. Third, taking out the function's "bend" via exponential (that is, ratio) factors, rather than via a polynomial technique (e.g., fitting a parabola), turns out to give an extraordinarily robust algorithm. In both reliability and speed, Ridders' method is generally competitive with the more highly developed and better established (but more complicated) method of van Wijngaarden, Dekker, and Brent, which we next discuss.

```
template <class T>
Doub zriddr(T &func, const Doub x1, const Doub x2, const Doub xacc) {
    Using Ridders' method, return the root of a function or functor func known to lie between x1
    and x2. The root will be refined to an approximate accuracy xacc.
    const Int MAXIT=60;
    Doub fl=func(x1);
    Doub fh=func(x2);
    if ((fl > 0.0 && fh < 0.0) || (fl < 0.0 && fh > 0.0)) {
        Doub xl=x1;
        Doub xh=x2;
        Doub ans=-9.99e99;
        for (Int j=0;j<MAXIT;j++) {
            Doub xm=0.5*(xl+xh);
            Doub fm=func(xm);
            Doub s=sqrt(fm*fm-fl*fh);
            if (s == 0.0) return ans;
            Doub xnew=xm+(xm-xl)*((fl >= fh ? 1.0 : -1.0)*fm/s);
            if (abs(xnew-ans) <= xacc) return ans;
            ans=xnew;
            Doub fnew=func(xnew);
            if (fnew == 0.0) return ans;
            if (SIGN(fm,fnew) != fm) {
                xl=xm;
                fl=fm;
                xh=ans;
                fh=fnew;
            } else if (SIGN(fl,fnew) != fl) {
                xh=ans;
                fh=fnew;
            } else if (SIGN(fh,fnew) != fh) {
                xl=ans;
                fl=fnew;
            } else throw("never get here.");
            if (abs(xh-xl) <= xacc) return ans;
        }
        throw("zriddr exceed maximum iterations");
    }
    else {
        if (fl == 0.0) return x1;
        if (fh == 0.0) return x2;
        throw("root must be bracketed in zriddr.");
    }
}
```

roots.h

Any highly unlikely value, to simplify logic below.

First of two function evaluations per iteration.

Updating formula.

Second of two function evaluations per iteration.

Bookkeeping to keep the root bracketed on next iteration.

CITED REFERENCES AND FURTHER READING:

Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §8.3.

- Ostrowski, A.M. 1966, *Solutions of Equations and Systems of Equations*, 2nd ed. (New York: Academic Press), Chapter 12.
- Ridders, C.J.F. 1979, "A New Algorithm for Computing a Single Root of a Real Continuous Function," *IEEE Transactions on Circuits and Systems*, vol. CAS-26, pp. 979–980.[1]

9.3 Van Wijngaarden-Dekker-Brent Method

While secant and false position formally converge faster than bisection, one finds in practice pathological functions for which bisection converges more rapidly. These can be choppy, discontinuous functions, or even smooth functions if the second derivative changes sharply near the root. Bisection always halves the interval, while secant and false position can sometimes spend many cycles slowly pulling distant bounds closer to a root. Ridders' method does a much better job, but it too can sometimes be fooled. Is there a way to combine superlinear convergence with the sureness of bisection?

Yes. We can keep track of whether a supposedly superlinear method is actually converging the way it is supposed to, and, if it is not, we can intersperse bisection steps so as to guarantee *at least* linear convergence. This kind of super-strategy requires attention to bookkeeping detail, and also careful consideration of how round-off errors can affect the guiding strategy. Also, we must be able to determine reliably when convergence has been achieved.

An excellent algorithm that pays close attention to these matters was developed in the 1960s by van Wijngaarden, Dekker, and others at the Mathematical Center in Amsterdam, and later improved by Brent [1]. For brevity, we refer to the final form of the algorithm as *Brent's method*. The method is *guaranteed* (by Brent) to converge, so long as the function can be evaluated within the initial interval known to contain a root.

Brent's method combines root bracketing, bisection, and *inverse quadratic interpolation* to converge from the neighborhood of a zero crossing. While the false-position and secant methods assume approximately linear behavior between two prior root estimates, inverse quadratic interpolation uses three prior points to fit an inverse quadratic function (x as a quadratic function of y) whose value at $y = 0$ is taken as the next estimate of the root x . Of course one must have contingency plans for what to do if the root falls outside of the brackets. Brent's method takes care of all that. If the three point pairs are $[a, f(a)]$, $[b, f(b)]$, $[c, f(c)]$, then the interpolation formula (cf. equation 3.2.1) is

$$x = \frac{[y - f(a)][y - f(b)]c}{[f(c) - f(a)][f(c) - f(b)]} + \frac{[y - f(b)][y - f(c)]a}{[f(a) - f(b)][f(a) - f(c)]} + \frac{[y - f(c)][y - f(a)]b}{[f(b) - f(c)][f(b) - f(a)]} \quad (9.3.1)$$

Setting y to zero gives a result for the next root estimate, which can be written as

$$x = b + P/Q \quad (9.3.2)$$

where, in terms of

$$R \equiv f(b)/f(c), \quad S \equiv f(b)/f(a), \quad T \equiv f(a)/f(c) \quad (9.3.3)$$

we have

$$\begin{aligned} P &= S [T(R - T)(c - b) - (1 - R)(b - a)] \\ Q &= (T - 1)(R - 1)(S - 1) \end{aligned} \quad (9.3.4)$$

In practice b is the current best estimate of the root and P/Q ought to be a “small” correction. Quadratic methods work well only when the function behaves smoothly; they run the serious risk of giving very bad estimates of the next root or causing machine failure by an inappropriate division by a very small number ($Q \approx 0$). Brent’s method guards against this problem by maintaining brackets on the root and checking where the interpolation would land before carrying out the division. When the correction P/Q would not land within the bounds, or when the bounds are not collapsing rapidly enough, the algorithm takes a bisection step. Thus, Brent’s method combines the sureness of bisection with the speed of a higher-order method when appropriate. We recommend it as the method of choice for general one-dimensional root finding where a function’s values only (and not its derivative or functional form) are available.

```
template <class T> roots.h
Doub zbrent(T &func, const Doub x1, const Doub x2, const Doub tol)
Using Brent's method, return the root of a function or functor func known to lie between x1
and x2. The root will be refined until its accuracy is tol.
{
    const Int ITMAX=100; Maximum allowed number of iterations.
    const Doub EPS=numeric_limits<Doub>::epsilon();
    Machine floating-point precision.
    Doub a=x1,b=x2,c=x2,d,e,fa=func(a),fb=func(b),fc,p,q,r,s,tol1,xm;
    if ((fa > 0.0 && fb > 0.0) || (fa < 0.0 && fb < 0.0))
        throw("Root must be bracketed in zbrent");
    fc=fb;
    for (Int iter=0;iter<ITMAX;iter++) {
        if ((fb > 0.0 && fc > 0.0) || (fb < 0.0 && fc < 0.0)) {
            c=a; Rename a, b, c and adjust bounding interval
            fc=fa; d.
            e=d=b-a;
        }
        if (abs(fc) < abs(fb)) {
            a=b;
            b=c;
            c=a;
            fa=fb;
            fb=fc;
            fc=fa;
        }
        tol1=2.0*EPS*abs(b)+0.5*tol; Convergence check.
        xm=0.5*(c-b);
        if (abs(xm) <= tol1 || fb == 0.0) return b;
        if (abs(e) >= tol1 && abs(fa) > abs(fb)) {
            s=fb/fa; Attempt inverse quadratic interpolation.
            if (a == c) {
                p=2.0*xm*s;
                q=1.0-s;
            } else {
                q=fa/fc;
                r=fb/fc;
                p=s*(2.0*xm*q*(q-r)-(b-a)*(r-1.0));
                q=(q-1.0)*(r-1.0)*(s-1.0);
            }
            if (p > 0.0) q = -q; Check whether in bounds.
            p=abs(p);
```

```

Doub min1=3.0*xm*q-abs(tol1*q);
Doub min2=abs(e*q);
if (2.0*p < (min1 < min2 ? min1 : min2)) {
    e=d;
    d=p/q;
    } else {
    d=xm;
    e=d;
    }
} else {
    d=xm;
    e=d;
}
a=b;
fa=fb;
if (abs(d) > tol1)
    b += d;
else
    b += SIGN(tol1,xm);
fb=func(b);
}
throw("Maximum number of iterations exceeded in zbrent");
}

```

Accept interpolation.

Interpolation failed, use bisection.

Bounds decreasing too slowly, use bisection.

Move last best guess to a.

Evaluate new trial root.

CITED REFERENCES AND FURTHER READING:

Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover), Chapters 3, 4.[1]

Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §7.2.

9.4 Newton-Raphson Method Using Derivative

Perhaps the most celebrated of all one-dimensional root-finding routines is *Newton's method*, also called the *Newton-Raphson method*. Joseph Raphson (1648–1715) was a contemporary of Newton who independently invented the method in 1690, some 20 years after Newton did, but some 20 years before Newton actually published it. This method is distinguished from the methods of previous sections by the fact that it requires the evaluation of both the function $f(x)$ and the derivative $f'(x)$, at arbitrary points x . The Newton-Raphson formula consists geometrically of extending the tangent line at a current point x_i until it crosses zero, then setting the next guess x_{i+1} to the abscissa of that zero crossing (see Figure 9.4.1). Algebraically, the method derives from the familiar Taylor series expansion of a function in the neighborhood of a point,

$$f(x + \delta) \approx f(x) + f'(x)\delta + \frac{f''(x)}{2}\delta^2 + \dots \quad (9.4.1)$$

For small enough values of δ , and for well-behaved functions, the terms beyond linear are unimportant, hence $f(x + \delta) = 0$ implies

$$\delta = -\frac{f(x)}{f'(x)} \quad (9.4.2)$$

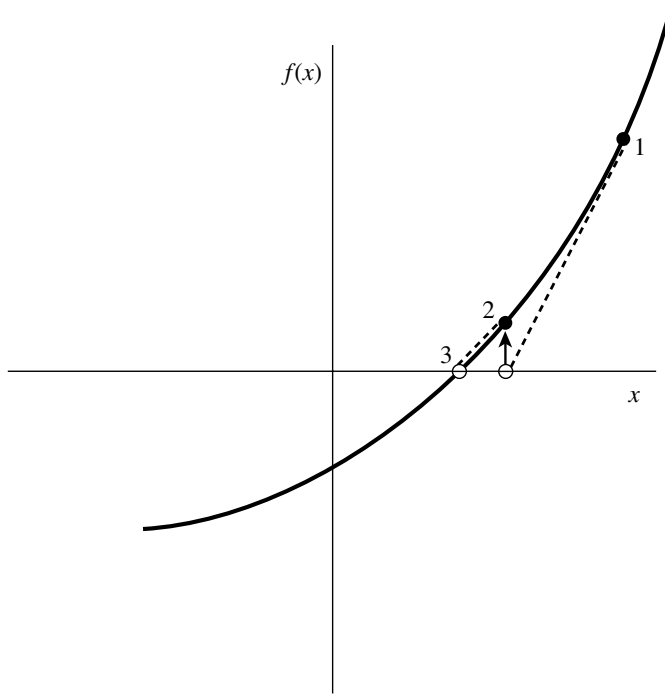


Figure 9.4.1. Newton's method extrapolates the local derivative to find the next estimate of the root. In this example it works well and converges quadratically.

Newton-Raphson is not restricted to one dimension. The method readily generalizes to multiple dimensions, as we shall see in §9.6 and §9.7, below.

Far from a root, where the higher-order terms in the series *are* important, the Newton-Raphson formula can give grossly inaccurate, meaningless corrections. For instance, the initial guess for the root might be so far from the true root as to let the search interval include a local maximum or minimum of the function. This can be death to the method (see Figure 9.4.2). If an iteration places a trial guess near such a local extremum, so that the first derivative nearly vanishes, then Newton-Raphson sends its solution off to limbo, with vanishingly small hope of recovery. Figure 9.4.3 demonstrates another possible pathology.

Why is Newton-Raphson so powerful? The answer is its rate of convergence: Within a small distance ϵ of x , the function and its derivative are approximately

$$\begin{aligned} f(x + \epsilon) &= f(x) + \epsilon f'(x) + \epsilon^2 \frac{f''(x)}{2} + \cdots, \\ f'(x + \epsilon) &= f'(x) + \epsilon f''(x) + \cdots \end{aligned} \quad (9.4.3)$$

By the Newton-Raphson formula,

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (9.4.4)$$

so that

$$\epsilon_{i+1} = \epsilon_i - \frac{f(x_i)}{f'(x_i)} \quad (9.4.5)$$

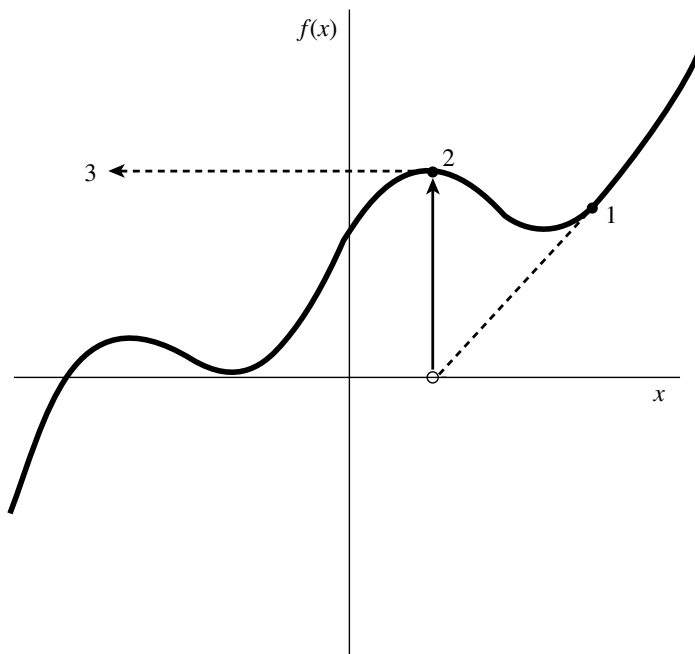


Figure 9.4.2. Unfortunate case where Newton's method encounters a local extremum and shoots off to outer space. Here bracketing bounds, as in `rtsafe`, would save the day.

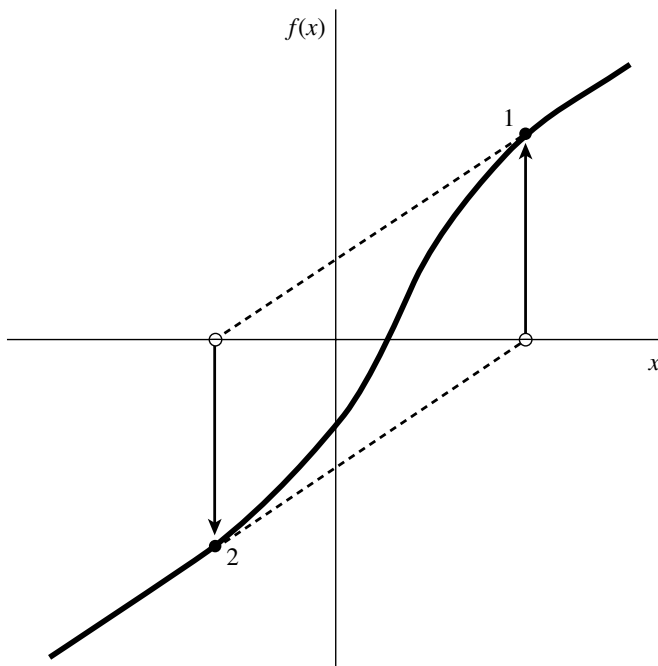


Figure 9.4.3. Unfortunate case where Newton's method enters a nonconvergent cycle. This behavior is often encountered when the function f is obtained, in whole or in part, by table interpolation. With a better initial guess, the method would have succeeded.

When a trial solution x_i differs from the true root by ϵ_i , we can use (9.4.3) to express $f(x_i)$, $f'(x_i)$ in (9.4.4) in terms of ϵ_i and derivatives at the root itself. The result is a recurrence relation for the deviations of the trial solutions

$$\epsilon_{i+1} = -\epsilon_i^2 \frac{f''(x)}{2f'(x)} \quad (9.4.6)$$

Equation (9.4.6) says that Newton-Raphson converges *quadratically* (cf. equation 9.2.3). Near a root, the number of significant digits approximately *doubles* with each step. This very strong convergence property makes Newton-Raphson the method of choice for any function whose derivative can be evaluated efficiently, and whose derivative is continuous and nonzero in the neighborhood of a root.

Even where Newton-Raphson is rejected for the early stages of convergence (because of its poor global convergence properties), it is very common to “polish up” a root with one or two steps of Newton-Raphson, which can multiply by two or four its number of significant figures.

For an efficient realization of Newton-Raphson, the user provides a routine that evaluates both $f(x)$ and its first derivative $f'(x)$ at the point x . The Newton-Raphson formula can also be applied using a numerical difference to approximate the true local derivative,

$$f'(x) \approx \frac{f(x + dx) - f(x)}{dx} \quad (9.4.7)$$

This is not, however, a recommended procedure for the following reasons: (i) You are doing two function evaluations per step, so *at best* the superlinear order of convergence will be only $\sqrt{2}$. (ii) If you take dx too small, you will be wiped out by roundoff, while if you take it too large, your order of convergence will be only linear, no better than using the *initial* evaluation $f'(x_0)$ for all subsequent steps. Therefore, Newton-Raphson with numerical derivatives is (in one dimension) always dominated by Brent’s method (§9.3). (In multidimensions, where there is a paucity of available methods, Newton-Raphson with numerical derivatives must be taken more seriously. See §9.6 – §9.7.)

The following routine invokes a user-supplied structure that supplies the function value and the derivative. The function value is returned in the usual way as a functor by overloading operator(), while the derivative is returned by the `df` function in the structure. For example, to find a root of the Bessel function $J_0(x)$ (derivative $-J_1(x)$) you would have a structure like

```
struct Functd {
    Bessjy bess;
    Doub operator() (const Doub x) {
        return bess.j0(x);
    }
    Doub df(const Doub x) {
        return -bess.j1(x);
    }
};
```

(While you can use any name for `Functd`, the name `df` is fixed.) Your code should then create an instance of this structure and pass it to `rtnewt`:

```
Functd fx;
Doub root=rtnewt(fx,x1,x2,xacc);
```

The routine `rtnewt` includes input bounds on the root `x1` and `x2` simply to be consistent with previous root-finding routines: Newton does not adjust bounds, and works only on local information at the point `x`. The bounds are used only to pick the mid-point as the first guess, and to reject the solution if it wanders outside of the bounds.

```
roots.h  template <class T>
        Doub rtnewt(T &funcd, const Doub x1, const Doub x2, const Doub xacc) {
        Using the Newton-Raphson method, return the root of a function known to lie in the interval
        [x1,x2]. The root will be refined until its accuracy is known within  $\pm xacc$ . funcd is a user-
        supplied struct that returns the function value as a functor and the first derivative of the function
        at the point x as the function df (see text).
        const Int JMAX=20;                               Set to maximum number of iterations.
        Doub rtn=0.5*(x1+x2);                             Initial guess.
        for (Int j=0;j<JMAX;j++) {
            Doub f=funcd(rtn);
            Doub df=funcd.df(rtn);
            Doub dx=f/df;
            rtn -= dx;
            if ((x1-rtn)*(rtn-x2) < 0.0)
                throw("Jumped out of brackets in rtnewt");
            if (abs(dx) < xacc) return rtn;               Convergence.
        }
        throw("Maximum number of iterations exceeded in rtnewt");
    }
```

While Newton-Raphson's global convergence properties are poor, it is fairly easy to design a fail-safe routine that utilizes a combination of bisection and Newton-Raphson. The hybrid algorithm takes a bisection step whenever Newton-Raphson would take the solution out of bounds, or whenever Newton-Raphson is not reducing the size of the brackets rapidly enough.

```
roots.h  template <class T>
        Doub rtsafe(T &funcd, const Doub x1, const Doub x2, const Doub xacc) {
        Using a combination of Newton-Raphson and bisection, return the root of a function bracketed
        between x1 and x2. The root will be refined until its accuracy is known within  $\pm xacc$ . funcd
        is a user-supplied struct that returns the function value as a functor and the first derivative of
        the function at the point x as the function df (see text).
        const Int MAXIT=100;                             Maximum allowed number of iterations.
        Doub xh,xl;
        Doub fl=funcd(x1);
        Doub fh=funcd(x2);
        if ((fl > 0.0 && fh > 0.0) || (fl < 0.0 && fh < 0.0))
            throw("Root must be bracketed in rtsafe");
        if (fl == 0.0) return x1;
        if (fh == 0.0) return x2;
        if (fl < 0.0) {                                    Orient the search so that  $f(x1) < 0$ .
            xl=x1;
            xh=x2;
        } else {
            xh=x1;
            xl=x2;
        }
        Doub rts=0.5*(x1+x2);                             Initialize the guess for root,
        Doub dxold=abs(x2-x1);                             the "stepsize before last,"
        Doub dx=dxold;                                     and the last step.
        Doub f=funcd(rts);
        Doub df=funcd.df(rts);
        for (Int j=0;j<MAXIT;j++) {                        Loop over allowed iterations.
            if (((rts-xh)*df-f)*((rts-xl)*df-f) > 0.0)    Bisect if Newton out of range,
                || (abs(2.0*f) > abs(dxold*df))) {        or not decreasing fast enough.
            }
```



```

    dxold=dx;
    dx=0.5*(xh-xl);
    rts=xl+dx;
    if (xl == rts) return rts;          Change in root is negligible.
} else {                               Newton step acceptable. Take it.
    dxold=dx;
    dx=f/df;
    Doub temp=rts;
    rts -= dx;
    if (temp == rts) return rts;
}
if (abs(dx) < xacc) return rts;        Convergence criterion.
Doub f=funcd(rts);
Doub df=funcd.df(rts);
The one new function evaluation per iteration.
if (f < 0.0)                           Maintain the bracket on the root.
    xl=rts;
else
    xh=rts;
}
throw("Maximum number of iterations exceeded in rtsafe");
}

```

For many functions, the derivative $f'(x)$ often converges to machine accuracy before the function $f(x)$ itself does. When that is the case one need not subsequently update $f'(x)$. This shortcut is recommended only when you confidently understand the generic behavior of your function, but it speeds computations when the derivative calculation is laborious. (Formally, this makes the convergence only linear, but if the derivative isn't changing anyway, you can do no better.)

9.4.1 Newton-Raphson and Fractals

An interesting sidelight to our repeated warnings about Newton-Raphson's unpredictable global convergence properties — its very rapid local convergence notwithstanding — is to investigate, for some particular equation, the set of starting values from which the method does, or doesn't, converge to a root.

Consider the simple equation

$$z^3 - 1 = 0 \quad (9.4.8)$$

whose single real root is $z = 1$, but which also has complex roots at the other two cube roots of unity, $\exp(\pm 2\pi i/3)$. Newton's method gives the iteration

$$z_{j+1} = z_j - \frac{z_j^3 - 1}{3z_j^2} \quad (9.4.9)$$

Up to now, we have applied an iteration like equation (9.4.9) only for real starting values z_0 , but in fact all of the equations in this section also apply in the complex plane. We can therefore map out the complex plane into regions from which a starting value z_0 , iterated in equation (9.4.9), will, or won't, converge to $z = 1$. Naively, we might expect to find a "basin of convergence" somehow surrounding the root $z = 1$. We surely do not expect the basin of convergence to fill the whole plane, because the plane must also contain regions that converge to each of the two complex roots. In fact, by symmetry, the three regions must have identical shapes. Perhaps they will be three symmetric 120° wedges, with one root centered in each?

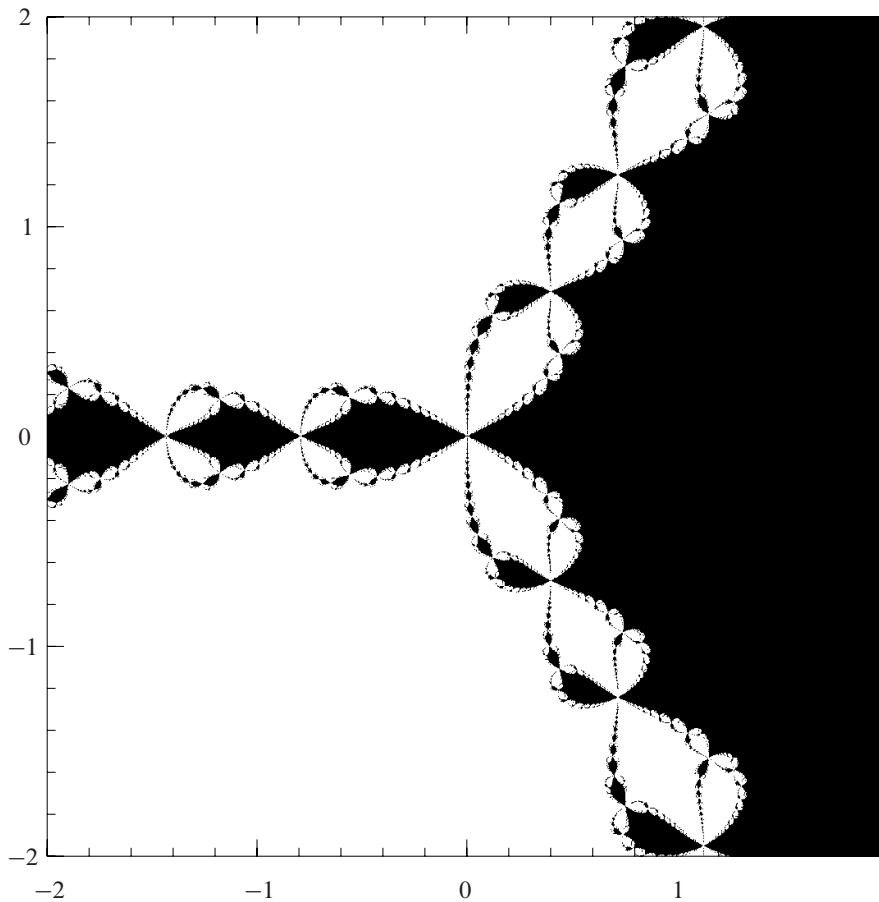


Figure 9.4.4. The complex z -plane with real and imaginary components in the range $(-2, 2)$. The black region is the set of points from which Newton's method converges to the root $z = 1$ of the equation $z^3 - 1 = 0$. Its shape is fractal.

Now take a look at Figure 9.4.4, which shows the result of a numerical exploration. The basin of convergence does indeed cover $1/3$ the area of the complex plane, but its boundary is highly irregular — in fact, *fractal*. (A fractal, so called, has self-similar structure that repeats on all scales of magnification.) How does this fractal emerge from something as simple as Newton's method and an equation as simple as (9.4.8)? The answer is already implicit in Figure 9.4.2, which showed how, on the real line, a local extremum causes Newton's method to shoot off to infinity. Suppose one is *slightly* removed from such a point. Then one might be shot off not to infinity, but — by luck — right into the basin of convergence of the desired root. But that means that in the neighborhood of an extremum there must be a tiny, perhaps distorted, copy of the basin of convergence — a kind of “one-bounce away” copy. Similar logic shows that there can be “two-bounce” copies, “three-bounce” copies, and so on. A fractal thus emerges.

Notice that, for equation (9.4.8), almost the whole real axis is in the domain of convergence for the root $z = 1$. We say “almost” because of the peculiar discrete points on the negative real axis whose convergence is indeterminate (see figure). What happens if you start Newton's method from one of these points? (Try it.)